

UNIVERSIDAD FIDELITAS
PROYECTO PROGRAMACIÓN BÁSICA



Grupo #1

Administrador de Tareas Personales

INTEGRANTES Carlos Amador

Issac Rodríguez

Josué Rivera

Óscar Saborío

Prof. Mario Granados Solís

Segundo Cuatrimestre 2025

Antecedentes

Los administradores de tareas personales son aplicaciones diseñadas para organizar, priorizar y dar seguimiento a las actividades diarias. Este tipo de herramienta ha ganado popularidad especialmente entre estudiantes, profesionales y personas que manejan múltiples responsabilidades. Su propósito principal es facilitar la planificación y ejecución de tareas para mejorar la productividad.

A continuación, según el sitio “Stel Order” explica la importancia del uso de un administrador de tareas en la actualidad:

“En un entorno cada vez más exigente y acelerado, debido a las nuevas tecnologías de la comunicación, la gestión eficiente del tiempo se ha convertido en una prioridad, tanto para el éxito profesional como para el bienestar personal. Afortunadamente, existen hoy en día una amplia variedad de plataformas y herramientas digitales que ayudan a organizar y priorizar las tareas diarias de manera más sencilla y eficiente”. (STEL Order, 2023)

Existen diferentes tipos de aplicación según la plataforma: algunas funcionan de manera local (sin necesidad de conexión a internet), otras son completamente en línea (requieren conexión constante), y muchas adoptan un enfoque mixto que permite sincronización entre dispositivos. Por ejemplo, Todoist es una aplicación multiplataforma que permite organizar tareas por proyectos, establecer fechas límite y sincronizar con herramientas como Google Calendar (Doist, s. f.). Por otro lado, Notion ofrece un espacio de trabajo integral donde se pueden gestionar tareas, tomar notas y crear bases de datos personalizadas, todo en una sola interfaz adaptable a distintas necesidades (Notion Labs Inc., s. f.).

Estas plataformas son utilizadas tanto a nivel individual como en entornos colaborativos, y pueden encontrarse como aplicaciones móviles, versiones web o software de escritorio. Su versatilidad y facilidad de uso las convierten en una opción clave para la gestión personal del tiempo y las responsabilidades.

Justificación

Hoy en día muchas personas tienen tantas cosas en su día a día que a veces se les olvida que era lo más importante o se les pasan tareas importantes. Esto puede ser una causa de estrés, desorganización y pérdida de tiempo. Por eso contar con una herramienta que ayude a organizar nuestras actividades diarias es algo muy útil.

Este proyecto busca crear un administrador de tareas personales, en otras palabras, una aplicación donde cualquier persona pueda anotar sus pendientes, organizar su día a día, poner fechas o recordatorios y ver su progreso. De esta forma, es más fácil mantenerse enfocado en lo más importante, cumplir con las responsabilidades y aprovechar mejor el tiempo.

Además, este proyecto es una gran oportunidad para poner en práctica lo que hemos aprendido en programación para desarrollar un software que realmente ayude a las

personas. También nos permite mejorar nuestras habilidades para planear, resolver problemas, trabajar de forma ordenada y en equipo.

Diseño preliminar

Administrador de Tareas Personales

Este diseño se ha planteado a partir de los hallazgos de la investigación sobre las características más valoradas en plataformas como Todoist y Notion. Se identificó que las funcionalidades más utilizadas son la organización por categorías, la sincronización multiplataforma, la posibilidad de asignar prioridades y el seguimiento del estado de las tareas. Asimismo, se valora la visualización de informes que permitan ver el progreso y el desempeño en el cumplimiento de tareas.

- **Módulo de inicio:** Inicio de sesión y despliegue de menú para acceder a los módulos. Funcionalidad: Inicio de sesión del usuario. Salida: Si el login es exitoso, se muestra el menú principal con acceso a los demás módulos.
- **Módulo 1 Registro de tareas:** Funcionalidad: Permitirá al usuario registrar nuevas tareas. Los datos que se podrán ingresar son: título, descripción, fecha de creación. Nota: No se asigna prioridad ni estado en este módulo. Si no se le asigna en la casilla de la tarea, saldrá el mensaje sin estado y sin prioridad.
- **Módulo 2 Asignación de prioridad y estado:** Funcionalidad: Permite modificar tareas existentes para asignar su prioridad: alta, media, baja. Estado: pendiente, en progreso, completada.
- **Módulo 3: Listado de tareas activas o pendientes:** Funcionalidad: Muestra tareas filtradas por estado (por ejemplo, solo las que están activas o pendientes).
- **Módulo 4: Módulo de informes:**
Informe 1: Porcentaje de tareas completadas.
Informe 2: Tareas más comunes por categoría o tipo.
Funcionalidad: Generación de reportes.
Sub informe: Porcentaje de tareas completadas y tareas más comunes por la categoría o tipos filtradas por estado (por ejemplo, solo las que están activas o pendientes y también las que ya han sido completadas).

Diseño en carpeta Proyecto

|

|----- [inicio.py](#) Inicio de sesión y menú.

|----- [menu.py](#) Menu: despliegue de menu.

|----- [registroTareas.py](#) módulo 1: Registro de tareas.

|----- [asignarEstado.py](#) módulo 2: Asignar prioridad y estado.

|----- [verTareas.py](#) módulo 3: Ver tareas activas o pendientes.

|----- [informes.py](#) módulo 4: Generación de informes.

|----- [variables.py](#) variables : Contiene todas las variables

Análisis del problema, Módulo inicio main

1. Comprensión del problema

¿Qué nos están pidiendo?	Crear un algoritmo de inicio de sesión donde se pregunte al usuario su nombre de usuario y contraseña para ingresar.
¿Qué elementos nos están dando?	nombres de usuario, con sus respectivas contraseñas
¿Qué se espera de resultado?	Se espera poder ingresar al sistema principal del programa.
¿Qué otra información necesitamos investigar?	No se requiere más información adicional para este problema, ya que es básico y todos los elementos están claramente definidos.

2. Identificación de requisitos

¿Cuáles son los datos de entrada?	Nombres de usuario y contraseñas
¿Qué estructuras se deben utilizar?	Entrada de datos, salida de datos, variables para almacenar los datos ingresados. if else
¿Cuáles son los cálculos esperados?	No se realiza ningún cálculo. Solo se almacenan y muestran los datos.
¿Cuáles son los datos de salida?	Ingreso al menú principal.

3. solución

Identifica si el problema puede dividirse en pasos o módulos	Modulos
Algoritmo Inicio Limpieza de la pantalla Al comenzar, el programa limpia la pantalla de la consola usando un comando específico para Windows (cls). Esto ayuda a mostrar la información de forma clara y ordenada desde el inicio. Importacion del menú principal El programa importa una función llamada menu desde otro archivo llamado menuPrincipal. Esta función se ejecutará cuando un usuario ingrese correctamente sus datos para mostrar las opciones principales del sistema.	

Almacenamiento de usuarios.

Los usuarios y sus contraseñas se guardan en una lista llamada usuarios. Cada elemento de esta lista es otra lista que contiene dos valores: el nombre del usuario y su clave.\

Bucle principal y menú de opciones.

El programa se ejecuta dentro de un ciclo infinito que muestra un menú con dos opciones:

- Registrar Usuario
 - Inicio de sesión
- El usuario debe ingresar el número correspondiente a la opción que desea elegir.

Validación de la opción ingresada.

Se verifica que la opción ingresada sea un número válido entre 0 y 9. Si no es así, se muestra un mensaje solicitando que solo se ingresen números, y se vuelve a mostrar el menú.

Registro de un nuevo usuario

Si el usuario elige la opción de registrar usuario:

- Se le solicita ingresar un nombre de usuario y una contraseña.
- Se revisa si ese nombre ya está registrado recorriendo la lista de usuarios.
- Si el nombre ya existe, se informa que no puede usarse y se solicita intentar con otro.
- Si no existe, se agrega el nuevo usuario con su contraseña a la lista y se confirma que el registro fue exitoso.

Inicio de sesión

Si el usuario elige la opción de inicio de sesión:

- Se permite hasta tres intentos para ingresar credenciales válidas.
- En cada intento se pide usuario y contraseña.
- Se busca en la lista si existe un usuario con esa contraseña.
- Si las credenciales son correctas, se muestra un mensaje de bienvenida y se llama a la función menu() para continuar con el sistema.
- Si las credenciales son incorrectas, se informa y se muestra cuántos intentos quedan.
- Si se supera el límite de tres intentos, se informa que el programa se cerrará y el ciclo principal termina.

Manejo de opciones inválidas

Si el usuario ingresa un número que no corresponde a ninguna opción válida (distinto de 1 o 2), el programa muestra un mensaje que indica que la opción no es válida y vuelve a mostrar el menú.

Fin

Análisis del problema, SubPrograma Menu

1. Comprensión del problema

¿Qué nos están pidiendo?	Crear un modulo con un menú principal que permita enviarnos al los subprogramas de registrar tareas, asignar estado, ver tareas, generar informes o salir del programa.
¿Qué elementos nos están dando?	Las opciones del menú y los módulos que se deben ejecutar según la selección: registroTareas, asignarEstado, verTareas, informes, main.
¿Qué se espera de resultado?	Que el usuario pueda interactuar con el menú y acceder a las funciones correctas, repitiendo el ciclo hasta que decida salir.
¿Qué otra información necesitamos investigar?	Los detalles de implementación interna de cada módulo (registroTareas, etc.) y cómo se almacenan las tareas.

2. Identificación de requisitos

¿Cuáles son los datos de entrada?	Opción del menú (SeleccionMenu) ingresada por el usuario.
¿Qué estructuras se deben utilizar?	Estructuras de decisión (if/else)
¿Cuáles son los cálculos esperados?	No hay cálculos matemáticos. Solo control de flujo y llamadas a módulos dependiendo de la opción elegida.

¿Cuáles son los datos de salida?	Resultados de cada módulo (registro, vista, informes, etc.) y mensajes en pantalla que guían al usuario.
----------------------------------	--

3. Estructuración de la solución

Identifica si el problema puede dividirse en pasos o módulos	Modulos
Algoritmo Inicio 1. Definir variables globales: - tareas, estadosContador, totalTareas, categoria1, categoria2, categoria3 para manejar información de las tareas y sus estados. 2. Iniciar un ciclo infinito: - Mantener el menú activo hasta que el usuario decida salir. 3. Mostrar el menú con opciones: 1. Registrar tarea 2. Asignar prioridad y estado 3. Ver tareas activas o pendientes 4. Ver informes 5. Salir del programa 4. Pedir al usuario que seleccione una opción. 5. Evaluar la opción ingresada: 1. Si es 1: mostrar mensaje “Registrar tarea” y llamar a la función registroTareasDef(). 2. Si es 2: mostrar mensaje “Asignar prioridad y estado” y llamar a la función seleccionAsignarEstado(). 3. Si es 3: mostrar mensaje “Ver tareas activas o pendientes” y llamar a la función verTareas(). 4. Si es 4: mostrar mensaje “Ver informes” y llamar a la función seleccionInformes(). 5. Si es 5: mostrar mensaje de despedida y salir del ciclo. 6. Si es cualquier otra opción: mostrar mensaje “Opción no válida” y repetir el ciclo. 6. Repetir el ciclo para seguir mostrando el menú hasta que el usuario seleccione salir. Fin	

Análisis del problema Módulo 1 registro tareas

1. Comprensión del problema

¿Qué nos están pidiendo?	Registrar una tarea ingresada por el usuario, incluyendo datos como nombre, categoría, y estado.
¿Qué elementos nos están dando?	Nombre de la tarea. Categoría o tipo. Estado de la tarea (pendiente, en proceso, completada).
¿Qué se espera de resultado?	Que la tarea se almacene correctamente en el sistema o lista de tareas.
¿Qué otra información necesitamos investigar?	Podría definirse si hay validaciones, como evitar campos vacíos o duplicados (esto dependerá del alcance del sistema).

2. Identificación de requisitos

¿Cuáles son los datos de entrada?	Nombre de la tarea, categoría, estado.
¿Qué estructuras se deben utilizar?	Variables para almacenar los datos. Arreglo o lista para guardar múltiples tareas. Condiciones si se desea validar la entrada.
¿Cuáles son los cálculos esperados?	No se requiere cálculo, solo almacenamiento y posible validación.
¿Cuáles son los datos de salida?	Mensaje de confirmación o error (si la tarea se registró correctamente o no).

3. Estructuración de la solución

Identifica si el problema puede dividirse en pasos o módulos	Modulos
Algoritmo Inicio Limpieza de la pantalla Al comenzar, el programa limpia la pantalla de la consola usando un comando específico para Windows (cls). Esto ayuda a mostrar la información de forma clara y ordenada desde el inicio. Importación de variables globales Se importa un módulo llamado variables que contiene estructuras de datos globales utilizadas para almacenar la lista de tareas, contadores y categorías. Definición de la función: registroTareasDef Esta función contiene la lógica para registrar nuevas tareas y se ejecuta hasta que el usuario decide salir. Declaración de variables globales dentro de la función Se especifica que ciertas variables (tareas, totalTareas, estadosContador, categoria1, categoria2, categoria3) usadas dentro de la función son globales, es decir, se encuentran definidas en el módulo variables y serán modificadas aquí.. Menú dentro de la función. Se ejecuta un ciclo while True que muestra un menú con dos opciones: • Registrar una nueva tarea • Volver al menú anterior (salir de esta función) • Validación de la opción seleccionada Se verifica que la opción ingresada sea un número válido entre 0 y 9 para evitar errores. Registro de una nueva tarea Si el usuario elige la opción 1: • Se crea una lista vacía llamada tarea para almacenar la información de la tarea. • Se genera un ID para la tarea basado en la cantidad actual de tareas (el siguiente número secuencial). • Se solicitan datos al usuario: título, descripción, fecha de creación, fecha de entrega, prioridad y categoría. • El estado inicial de la tarea se fija como "Activa". • Según la categoría seleccionada (1, 2 o 3), se asigna el nombre correspondiente y se incrementa el contador de esa categoría. Si la categoría es inválida, se asigna un texto indicándolo. • Se agregan todos estos datos a la lista tarea en orden. • La tarea se añade a la lista global variables.tareas. • Se incrementa el contador total de tareas y un contador asociado al estado "Activa" en la variable	

estadosContador.

- Finalmente, se muestra un resumen en pantalla confirmando que la tarea fue registrada, detallando todos sus atributos.

Salir del menú de registro

Si el usuario elige la opción 2, la función termina y se regresa al menú o módulo que la llamó.

Fin

Análisis del problema, Módulo 2 asignarEstado

1. Comprensión del problema

¿Qué nos están pidiendo?	Diseñar un submenú que permita cambiar el estado o prioridad de una tarea, o volver al menú principal.
¿Qué elementos nos están dando?	Se dan tres módulos: cambiarEstado, cambiarPrioridad y Menu, además de la opción de entrada SeleccionMenu.
¿Qué se espera de resultado?	Que el usuario pueda seleccionar una opción del submenú y ejecutar correctamente la acción elegida.
¿Qué otra información necesitamos investigar?	Cómo se identifican las tareas para cambiarles estado o prioridad, y cómo se almacenan los cambios.

2. Identificación de requisitos

¿Cuáles son los datos de entrada?	La opción del submenú ingresada por el usuario (SeleccionMenu).
¿Qué estructuras se deben utilizar?	Estructuras condicionales (if/else) para decidir qué acción tomar.
¿Cuáles son los cálculos esperados?	No se requiere cálculo matemático, solo control de flujo.
¿Cuáles son los datos de salida?	Ejecución de la acción deseada: cambiar estado, prioridad o volver al menú.

3. Estructuración de la solución

Identifica si el problema puede dividirse en pasos o módulos	Modulo
Algoritmo Inicio Función: seleccionAsignarEstado() 1. Iniciar un ciclo infinito. 2. Mostrar el menú con opciones: 1. Cambiar estado de la tarea 2. Cambiar prioridad de la tarea 3. Volver al menú anterior (salir del ciclo) 3. Pedir al usuario que ingrese una opción. 4. Validar que la opción ingresada sea un número entre 1 y 3: ○ Si no es un número válido, mostrar mensaje de error y repetir el paso 3. 5. Según la opción elegida: ○ Si es 1: llamar a la función cambiarEstado(). ○ Si es 2: llamar a la función cambiarPrioridad(). ○ Si es 3: salir del ciclo y volver al menú anterior. 6. Si la opción es distinta a las anteriores, mostrar mensaje de "Opción no válida" y repetir el ciclo. Función: cambiarEstado()	

1. Verificar si la lista tareas está vacía:
 - Si está vacía, mostrar "No hay tareas registradas" y terminar la función.
2. Mostrar la lista de tareas registradas con su número, título y estado actual.
3. Solicitar al usuario que seleccione el número de la tarea a la que desea cambiar el estado.
4. Buscar la tarea en la lista que coincida con el número ingresado.
5. Si no se encuentra la tarea, mostrar mensaje "No se encontró una tarea con ese número" y terminar la función.
6. Si se encuentra la tarea:
 - a. Mostrar opciones de estado para cambiar:
 1. Pendiente
 2. Activa
 3. Completada
 - b. Pedir al usuario que seleccione el nuevo estado.
7. Determinar el índice del estado actual de la tarea (idActual):
 - 0 para "Pendiente"
 - 1 para "Activa"
 - 2 para "Completada"
 - -1 si no coincide con ninguno.
8. Calcular el índice del nuevo estado (idNuevo) restando 1 a la opción seleccionada.
9. Verificar que el nuevo estado sea diferente al actual y que el índice sea válido (0,1 o 2).
 - Si no, mostrar mensaje que la tarea ya tiene ese estado o la opción es inválida.
10. Si es válido:
 - a. Restar 1 al contador del estado actual en estadosContador.
 - b. Sumar 1 al contador del nuevo estado en estadosContador.
 - c. Actualizar el estado de la tarea al nuevo estado.
 - d. Mostrar mensaje confirmando el cambio de estado.
11. Terminar la función.

Función: cambiarPrioridad()

1. Verificar si la lista tareas está vacía:
 - Si está vacía, mostrar "No hay tareas registradas" y terminar la función.
2. Mostrar la lista de tareas con su número, título y prioridad actual.
3. Solicitar al usuario que seleccione el número de la tarea a la que desea cambiar la prioridad.
4. Buscar la tarea en la lista que coincida con el número ingresado.
5. Si no se encuentra la tarea, mostrar mensaje "No se encontró una tarea con ese número" y terminar la función.
6. Si se encuentra la tarea:
 - a. Mostrar opciones de prioridad para cambiar:
 1. Baja
 2. Media
 3. Alta
 - b. Pedir al usuario que seleccione la nueva prioridad.
7. Asignar la nueva prioridad basada en la opción seleccionada.
 - Si la opción no es válida, asignar cadena vacía.
8. Verificar que la nueva prioridad sea válida y diferente a la actual.
 - Si no, mostrar mensaje correspondiente (opción inválida o ya tiene esa prioridad).
9. Si es válida, actualizar la prioridad de la tarea y mostrar mensaje confirmando el cambio.
10. Terminar la función.

fin

Análisis del problema, Módulo 3 listadoTareas

1. Comprensión del problema

¿Qué nos están pidiendo?	Listado de tareas con la condición de que el estado de la tarea sea Activa o pendiente
¿Qué elementos nos están dando?	contador, descripción de la tarea, fecha de creación, fecha de entrega, categoría de la tarea, prioridad y estado de la tarea
¿Qué se espera de resultado?	Mostrar los elementos que conforman la información de la tarea
¿Qué otra información necesitamos investigar?	No se debe investigar más

2. Identificación de requisitos

¿Cuáles son los datos de entrada?	contador, descripción de la tarea, fecha de creación, fecha de entrega, categoría de la tarea, prioridad y estado de la tarea
¿Qué estructuras se deben utilizar?	if else, outputs.
¿Cuáles son los cálculos esperados?	ninguno
¿Cuáles son los datos de salida?	Mostrar el listado de tareas activas o pendientes.

3. Estructuración de la solución

Identifica si el problema puede dividirse en pasos o módulos	modulos
Algoritmo Inicio Limpieza de pantalla Al iniciar, el programa limpia la consola usando el comando cls para Windows, dejando la pantalla ordenada para mostrar el menú y los resultados. Importación de variables globales Se importan desde el módulo variables varias estructuras y contadores relacionados con las tareas: • tareas: la lista principal que contiene todas las tareas registradas. • totalTareas, estadosContador, categoria1, categoria2, categoria3: variables que almacenan totales y contadores de tareas por estado y categoría. Función seleccionVerTareas Esta función muestra un menú simple que permite al usuario elegir entre: • Ver las tareas pendientes o activas. • Regresar al menú anterior (salir de esta función). El menú se repite en un ciclo infinito hasta que el usuario decida salir. Se valida la opción ingresada, mostrando un mensaje si es inválida. Función verTareas Esta función recorre la lista completa de tareas (tareas) y muestra en pantalla solo aquellas cuyo estado no sea "Completada". Esto significa que se muestran las tareas activas o pendientes. Función verTareas Esta función recorre la lista completa de tareas (tareas) y muestra en pantalla solo aquellas cuyo estado no sea "Completada". Esto significa que se muestran las tareas activas o pendientes. Para cada tarea mostrada, imprime: • Número o ID de tarea • Título • Descripción • Estado actual • Prioridad asignada • Categoría a la que pertenece • Fechas de creación y entrega Además, lleva un contador interno de cuántas tareas se han mostrado. Mensaje cuando no hay tareas pendientes Si al finalizar el recorrido no se encontró ninguna tarea pendiente o activa, el programa informa que no hay tareas para mostrar. Fin	

Análisis del problema, Módulo 4 informes menu

4. Comprensión del problema

¿Qué nos están pidiendo?	Crear un submenú que permita generar informes sobre el porcentaje de tareas completadas, tareas más comunes por categoría o tipo, o volver al menú principal.
¿Qué elementos nos están dando?	Opciones del submenú (1, 2, 3) y tres módulos: Informe1, Informe2 y Menu.
¿Qué se espera de resultado?	Mostrar el informe correspondiente o regresar al menú principal según la opción seleccionada.
¿Qué otra información necesitamos investigar?	Cómo se calculan los porcentajes y qué criterios se usan para determinar las tareas más comunes.

5. Identificación de requisitos

¿Cuáles son los datos de entrada?	La opción del submenú ingresada por el usuario (SeleccionMenu).
¿Qué estructuras se deben utilizar?	Estructuras condicionales (if/else o switch) para ejecutar el módulo correspondiente.
¿Cuáles son los cálculos esperados?	Cálculo del porcentaje de tareas completadas y análisis de frecuencia de tareas por categoría o tipo.
¿Cuáles son los datos de salida?	Informes generados por los módulos Informe1 o Informe2, o retorno al menú principal.

6. Estructuración de la solución

Identifica si el problema puede dividirse en pasos o módulos	Modulos
--	---------

Algoritmo

Inicio

Función: seleccionInformes()

1. Iniciar un ciclo infinito.
2. Mostrar el menú con opciones:
 1. Informe de tareas completadas
 2. Informe de tareas por categoría
 3. Volver al menú anterior (salir del ciclo)
3. Solicitar al usuario que ingrese una opción numérica.
4. Evaluar la opción ingresada:
 - Si es 1: llamar a la función informe1().
 - Si es 2: llamar a la función informe2().
 - Si es 3: salir del ciclo y volver al menú anterior.
 - Si es cualquier otra opción, mostrar mensaje "Opción no válida".
5. Repetir el ciclo para seguir mostrando el menú.

Función: informe1()

1. Mostrar la cantidad total de tareas creadas (variable variables.totalTareas).
2. Mostrar la cantidad de tareas completadas (variable variables.estadosContador[2][1]).
3. Verificar si hay tareas creadas (variables.totalTareas > 0):
 - Si sí, calcular el porcentaje de tareas completadas como:
 $(\text{cantidad de tareas completadas} / \text{total de tareas}) * 100$
y mostrarlo con el símbolo %.
 - Si no, mostrar el mensaje "No hay tareas registradas aún."

Función: informe2()

1. Mostrar la cantidad de tareas con categoría "Trabajo/Estudio" (variable variables.categoria1).
2. Mostrar la cantidad de tareas con categoría "Personal/Hogar" (variable variables.categoria2).
3. Mostrar la cantidad de tareas con categoría "Salud/Bienestar" (variable variables.categoria3).

Fin

Análisis del problema, Módulo 4 informe 1

1. Comprensión del problema

¿Qué nos están pidiendo?	Calcular el porcentaje de tareas completadas y mostrarlo al usuario.
¿Qué elementos nos están dando?	La cantidad total de tareas (TotalTareas) y la cantidad de tareas completadas (TareasCompletadas).
¿Qué se espera de resultado?	Un mensaje que indique el porcentaje de tareas completadas o una advertencia si no hay suficientes tareas.
¿Qué otra información necesitamos investigar?	Validaciones adicionales necesarias y formato de salida del porcentaje.

2. Identificación de requisitos

¿Cuáles son los datos de entrada?	TotalTareas y TareasCompletadas, ingresadas por el usuario.
¿Qué estructuras se deben utilizar?	Condicionales para verificar que TotalTareas sea mayor a 0, y una operación aritmética para el cálculo.
¿Cuáles son los cálculos esperados?	$(\text{TareasCompletadas} / \text{TotalTareas}) * 100$ para obtener el porcentaje.
¿Cuáles son los datos de salida?	El porcentaje de tareas completadas o el mensaje de advertencia si no hay suficientes tareas.

3. Estructuración de la solución

Identifica si el problema puede dividirse en pasos o módulos	Modulos
Algoritmo Inicio Función: seleccionInformes() 1. Iniciar un ciclo infinito para mantener activo el menú de reportes. 2. Mostrar el menú con opciones: 1) Informe tareas completadas. 2) Informe tareas por categoría. 3) Menú (volver al menú anterior / salir del ciclo). 3. Pedir al usuario que ingrese una opción (se lee como número entero). 4. Según la opción elegida: - Si es 1: llamar a la función informe1(). - Si es 2: llamar a la función informe2(). - Si es 3: salir del ciclo y volver al menú anterior. - En cualquier otro caso: mostrar mensaje "Opción no válida." y repetir el ciclo. 5. Terminar cuando el usuario elige la opción 3. --- Función: informe1() (Porcentaje de tareas completadas) Usa variables definidas en el módulo variables: - variables.totalTareas → total de tareas registradas. - variables.estadosContador → matriz/lista de estados con sus conteos; en el índice [2][1] está el número de tareas completadas. 1. Verificar si hay tareas registradas ($\text{variables.totalTareas} > 0$). - Si no hay: mostrar "No hay tareas registradas aún." y terminar la función.	

2. Si sí hay:

- a) Tomar el número de completadas desde `variables.estadosContador[2][1]`.
- b) Calcular el porcentaje: $(\text{completadas} / \text{totalTareas}) \times 100$
- c) Mostrar: "Porcentaje de tareas completadas: X %".

3. Terminar la función.

Función: `informe2()` (Cantidad de tareas por categoría)

Usa contadores por categoría en variables:

- `variables.categoria1` → Trabajo/Estudio.
- `variables.categoria2` → Personal/Hogar.
- `variables.categoria3` → Salud/Bienestar.

1. Imprimir un espacio en blanco para separar el informe.
2. Mostrar: "La cantidad de tareas con categoría Trabajo/Estudio son {categoria1}".
3. Imprimir un espacio en blanco.
4. Mostrar: "La cantidad de tareas con categoría Personal/Hogar son {categoria2}".
5. Imprimir un espacio en blanco.
6. Mostrar: "La cantidad de tareas con categoría Salud/Bienestar son {categoria3}".
7. Imprimir un espacio en blanco y terminar la función.

Fin

Análisis del problema

1. Comprensión del problema

¿Qué nos están pidiendo?	Tareas más comunes por categoría o tipo.
¿Qué elementos nos están dando?	categoría1, 2 y 3
¿Qué se espera de resultado?	Muestre cuantas tareas hay de cada categoría
¿Qué otra información necesitamos investigar?	no hay nada mas que investigar

2. Identificación de requisitos

¿Cuáles son los datos de entrada?	categoría 1, categoría2 y categoría3
¿Qué estructuras se deben utilizar?	outputs
¿Cuáles son los cálculos esperados?	Ninguno

¿Cuáles son los datos de salida?	El conteo cuentas tareas están en categoría 1, 2 y 3
----------------------------------	--

3. Estructuración de la solución

Identifica si el problema puede dividirse en pasos o módulos	módulos
Algoritmo - informes.py Inicio Función: seleccionInformes() 1. Iniciar un ciclo infinito para mantener activo el menú de reportes. 2. Mostrar el menú con opciones: 1) Informe tareas completadas. 2) Informe tareas por categoría. 3) Menú (volver al menú anterior / salir del ciclo). 3. Pedir al usuario que ingrese una opción (se lee como número entero). 4. Según la opción elegida: - Si es 1: llamar a la función informe1(). - Si es 2: llamar a la función informe2(). - Si es 3: salir del ciclo y volver al menú anterior. - En cualquier otro caso: mostrar mensaje "Opción no válida." y repetir el ciclo. a 5. Terminar cuando el usuario elige la opción 3. --- Función: informe1() (Porcentaje de tareas completadas) Usa variables definidas en el módulo variables: - variables.totalTareas → total de tareas registradas. - variables.estadosContador → matriz/lista de estados con sus conteos; en el índice [2][1] está el número de tareas completadas. 1. Verificar si hay tareas registradas (variables.totalTareas > 0). - Si no hay: mostrar "No hay tareas registradas aún." y terminar la función. 2. Si sí hay: a) Tomar el número de completadas desde variables.estadosContador[2][1]. b) Calcular el porcentaje: (completadas / totalTareas) x 100 c) Mostrar: "Porcentaje de tareas completadas: X %". 3. Terminar la función. --- Función: informe2() (Cantidad de tareas por categoría) Usa contadores por categoría en variables:	

- variables.categoria1 → Trabajo/Estudio.
- variables.categoria2 → Personal/Hogar.
- variables.categoria3 → Salud/Bienestar.

1. Imprimir un espacio en blanco para separar el informe.
2. Mostrar: "La cantidad de tareas con categoría Trabajo/Estudio son {categoria1}".
3. Imprimir un espacio en blanco.
4. Mostrar: "La cantidad de tareas con categoría Personal/Hogar son {categoria2}".
5. Imprimir un espacio en blanco.
6. Mostrar: "La cantidad de tareas con categoría Salud/Bienestar son {categoria3}".
7. Imprimir un espacio en blanco y terminar la función.

Bibliografía

Doist. (s. f.). Todoist: The to-do list to organize work & life. <https://todoist.com>

Notion Labs Inc. (s. f.). Notion – One workspace. Every team. <https://www.notion.so>

STEL Order. (2023). Importancia del uso de un administrador de tareas. <https://www.stelorder.com>